

EXPERIMENT:17

Implement Mobile Price Prediction using appropriate machine learning algorithm

Code:

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
from google.colab import files
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

Step 2: Upload Dataset

```
print("Upload train.csv")
```

```
uploaded = files.upload()
```

Step 3: Read Dataset

```
df = pd.read_csv("train.csv")
```

Step 4: Display Dataset

```
print("\nFirst Five Rows:")
```

```
print(df.head())
```

```
print("\nDataset Shape:")
```

```
print(df.shape)
```

```
print("\nDataset Information:")
```

```
print(df.info())
```

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

Step 5: Separate Features and Target

```
X = df.drop("price_range", axis=1)
```

```
y = df["price_range"]
```

Step 6: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```

y,
test_size=0.2,
random_state=42,
stratify=y
)
# Step 7: Train Model
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)
model.fit(X_train, y_train)
# Step 8: Prediction
y_pred = model.predict(X_test)
# Step 9: Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nModel Accuracy: {:.2f}%".format(accuracy * 100))
# Step 10: Classification Report
print("\nClassification Report")
print(classification_report(y_test, y_pred))
# Step 11: Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["0", "1", "2", "3"],
    yticklabels=["0", "1", "2", "3"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")

```

```

plt.title("Confusion Matrix")
plt.show()
# Step 12: Feature Importance
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})
importance = importance.sort_values(
    by="Importance",
    ascending=False
)
print("\nTop Feature Importance")
print(importance)
# Plot Feature Importance
plt.figure(figsize=(10,6))
plt.barh(
    importance["Feature"],
    importance["Importance"]
)
plt.gca().invert_yaxis()
plt.title("Feature Importance")
plt.xlabel("Importance")
plt.show()
# Step 13: Predict New Sample
sample = X.iloc[[0]]
prediction = model.predict(sample)
print("\nPrediction for First Mobile:")
print("Predicted Price Range =", prediction[0])
# Price Range Meaning
price_dict = {
    0: "Low Cost",
    1: "Medium Cost",

```

```

2: "High Cost",
3: "Very High Cost"
}

print("Category:", price_dict[prediction[0]])

```

Output:

```

... Upload train.csv
  Choose Files train[1].csv
train[1].csv(text/csv) - 122403 bytes, last modified: 8/7/2026 - 100% done
Saving train[1].csv to train[1] (2).csv
Uploaded File: train[1] (2).csv

First 5 Rows:
  battery_power  blue  clock_speed  dual_sim  fc  four_g  int_memory  m_dep  \
0             842    0           2.2         0    1         0           7    0.6
1            1021    1           0.5         1    0         1          53    0.7
2             563    1           0.5         1    2         1          41    0.9
3             615    1           2.5         0    0         0          10    0.8
4            1821    1           1.2         0   13         1          44    0.6

  mobile_wt  n_cores  ...  px_height  px_width  ram  sc_h  sc_w  talk_time  \
0         188        2  ...         20       756  2549    9    7          19
1         136        3  ...         905     1988  2631   17    3           7
2         145        5  ...        1263     1716  2603   11    2           9
3         131        6  ...        1216     1786  2769   16    8          11
4         141        2  ...        1208     1212  1411    8    2          15

  three_g  touch_screen  wifi  price_range
0         0             0     1             1
1         1             1     0             2
2         1             1     0             2
3         1             0     0             2
4         1             1     0             1

[5 rows x 21 columns]

... [5 rows x 21 columns]

Dataset Shape:
(2000, 21)

Column Names:
['battery_power', 'blue', 'clock_speed', 'dual_sim', 'fc', 'four_g', 'int_memory', 'm_dep', 'mobile_wt',

Missing Values:
battery_power    0
blue             0
clock_speed      0
dual_sim         0
fc               0
four_g           0
int_memory       0
m_dep            0
mobile_wt        0
n_cores          0
pc               0
px_height        0
px_width         0
ram              0
sc_h             0
sc_w             0
talk_time        0
three_g          0
touch_screen     0
wifi             0

```

```

... price_range      0
dtype: int64

=====
Model Accuracy
=====
87.75%

=====
Classification Report
=====

```

	precision	recall	f1-score	support
0	0.94	0.95	0.95	100
1	0.81	0.84	0.82	100
2	0.82	0.79	0.81	100
3	0.94	0.93	0.93	100
accuracy			0.88	400
macro avg	0.88	0.88	0.88	400
weighted avg	0.88	0.88	0.88	400

```

=====
Confusion Matrix
=====
[[95  5  0  0]
 [ 6 84 10  0]
 [ 0 15 79  6]
 [ 0  0  7 93]]

```

```

... =====
Feature Importance
=====

```

	Feature	Importance
13	ram	0.482913
0	battery_power	0.072960
12	px_width	0.057762
11	px_height	0.055650
8	mobile_wt	0.038956
6	int_memory	0.035032
16	talk_time	0.030427
10	pc	0.029901
15	sc_w	0.028026
2	clock_speed	0.027638
14	sc_h	0.027481
4	fc	0.025799
7	m_dep	0.025140
9	n_cores	0.023310
5	four_g	0.007000
18	touch_screen	0.006984
1	blue	0.006976
3	dual_sim	0.006826
19	wifi	0.006460
17	three_g	0.004760

```

=====
Prediction
=====
Predicted Price Range: 1
Category: Medium Cost

```